# 命令行程序解释

DeepSeek 对话完整截图记录

---

学号姓名：202600130097 蔡昀晓

项目名称：Openlab 初试

记录来源：DeepSeek 安卓客户端 · 会话「命令行程序解释」

整理日期：2026 年 10 月 25 日

页数：共 18 页（含封面）

说明：本文档按会话从上到下的原有顺序排列，内容为原界面截图，
未对 AI 回答的文字与代码作任何改写或删减；代码块中超出屏幕
右边缘的部分在原客户端中即被截断，并非本文档裁切所致。

你能给我解释一下什么是命令行程序吗

命令行程序，简单说就是**通过输入文字指令来操作的程序**，没有图形界面（按钮、菜单那些），全靠打字和它交流。

可以这样理解：

- **图形界面程序**：像点外卖 App，你点按钮、滑菜单。
- **命令行程序**：像给外卖店发短信，你打一行字“我要一份牛肉面”，它回你文字结果。

**常见例子：**

- Windows 的 `cmd`、PowerShell
- macOS/Linux 的 `Terminal`（终端）
- 具体命令如 `ls`（列出文件）、`cd`（切换目录）、`git`（版本控制）、`python`（运行 Python）

**核心特点：**

1. **纯文本交互**：输入命令，输出文字结果。
2. **效率高**：熟练后比点鼠标快，还能批量操作。

2. **效率高**：熟练后比点鼠标快，还能批量操作。

3. **可脚本化**：能把多条命令写成脚本自动执行。

4. **资源占用小**：不需要图形界面。

**一个直观例子：**
在终端输入 `ls`，它列出当前文件夹的所有文件；输入 `cd Documents`，就进入 Documents 文件夹。整个过程就是“打字 → 回车 → 看文字反馈”。

如果你刚开始接触，可以把它想象成**和电脑用文字对话**，而不是用鼠标点来点去。需要我举个具体命令的例子帮你理吗?

那请你帮我写一个命令行待办清单，要求不需要图形界面 核心逻辑得能跑通

已思考（用时 6 秒） >

下面是一个完整可运行的 Python 命令行待办清单。用 `argparse` 做子命令，数据存成 JSON 文件，无需任何图形界面，也无需第三方库。

## 代码（保存为 `todo.py`）

python 复制

```python
#!/usr/bin/env python3
"""极简命令行待办清单"""

import argparse
import json
import os
import sys
from datetime import datetime

DATA_FILE = os.path.join(os.path.expanduser("~"), ".todo.

def load_tasks():
    if not os.path.exists(DATA_FILE):
        return []
    try:
        with open(DATA_FILE, "r", encoding="utf-8") a
            return json.load(f)
    except (json.JSONDecodeError, OSError):
        return []

def save_tasks(tasks):
    with open(DATA_FILE, "w", encoding="utf-8") as f
        json.dump(tasks, f, ensure_ascii=False, indent 2)
    tasks.append({
        "text": args.text,
        "done": False,
        "created": datetime.now().strftime("%Y-%m-%d %H:%
    })
    save_tasks(tasks)
    print(f"已添加: {args.text}")
```

```
    save_tasks(tasks)
    print(f"已添加: {args.text}")


def cmd_list(args):  # 显示 [ done ] 或 [ ]


    l = sub.add_parser("list", help="列出待办")
    l.add_argument("-a", "--all", action="store_true", he
    l.set_defaults(func=cmd_list)

    for name, func, helptext in [
        ("done", cmd_done, "标记完成"),

    c = sub.add_parser("clear", help="清空全部")
    c.set_defaults(func=cmd_clear)

    args = p.parse_args()
    args.func(args)
```

## 设计说明

- **数据持久化**：存到用户主目录下的 `~/.todo.json`，关掉终端再打开数据还在。
- **子命令结构**：`add` / `list` / `done` / `undone` / `rm` / `clear`，每个命令只做一件事，便于扩展。
- **ID 即序号**：直接对应 `list` 显示的数字，简单直观（注意删除后 ID 会重新编号）。
- **只用标准库**：`argparse` + `json` + `os`，任何 Python 3.7+ 环境都能跑。

- **只用标准库**：`argparse` + `json` + `os`，任何 Python 3.7+ 环境都能跑。

## 小提示

- Windows 终端如果中文乱码，先执行 `chcp 65001` 切到 UTF-8。
- 想变成全局命令，可以 `chmod +x todo.py` 后加个 alias，例如 `alias todo='python3 /path/to/todo.py'`，之后直接 `todo add "xxx"` 即可。

需要我再加个**截止日期**或**优先级**字段吗？逻辑上只是多存两个字段、排序时多一步而已。

> 待办事项的删除 优先级 替换截止日期都有吗 没的话把这些功能加上

已思考（用时 12 秒） >

三个功能都有：**删除**原来就有（`rm`），**优先级**和**截止日期**这次加上，并且顺手加了 `edit` 来修改它们、`list` 按优先级+截止日期排序、逾期高亮。下面是完整替换版。

## 代码（保存为 `todo.py`）

# 代码（保存为 todo.py）

python 复制

```python
#!/usr/bin/env python3
"""命令行待办清单：支持优先级、截止日期、删除、编辑"""

import argparse
import json
import os
import sys
from datetime import datetime, date

DATA_FILE = os.path.join(os.path.expanduser("~"), ".todo.

PRIORITY_ORDER = {"high": 0, "medium": 1, "low": 2}
PRIORITY_LABEL = {"high": "高", "medium": "中", "low"


# ---------- 数据层 ----------

def load_tasks():
    if not os.path.exists(DATA_FILE):
        return []
    try:
        with open(DATA_FILE, "r", encoding="utf-8") as f:
            tasks = json.load(f)
    except (json.JSONDecodeError, OSError):
        return []

    # 兼容旧数据：补字段、补 ID
    nxt = max((t.get("id") or 0 for t in tasks), defa
    for t in tasks:
        t.setdefault("done", False)
```

```
        t.setdefault("done", False)
        t.setdefault("priority", "medium")
        t.setdefault("due", None)
        if not t.get("id"):
            t["id"] = nxt
            nxt += 1
    return tasks

    print(f"{'ID':<5}{'状态':<6}{'优先级':<8}{'截止':<16}内
    print("-" * 56)
    for t in items:
        mark = "[x]" if t["done"] else "[ ]"
        pr = PRIORITY_LABEL.get(t["priority"], "中")
        due = t.get("due") or "-"
        changed = True
    if hasattr(args, "due"):
        t["due"] = args.due
    a = sub.add_parser("add", help="添加待办")
    a.add_argument("text", help="待办内容")
    a.add_argument("-p", "--priority", choices=["high", "
                   default="medium", help="优先级，默认 mec
    a.add_argument("-d", "--due", type=parse_due, default
                   help="截止日期 YYYY-MM-DD")
    a.set_defaults(func=cmd_add)

    l = sub.add_parser("list", help="列出待办（按优先级+截止
    l.add_argument("-a", "--all", action="store_true", he
    l.set_defaults(func=cmd_list)

    for name, func, helptext in [
        ("done", cmd_done, "标记完成"),
        ("undone", cmd_undone, "标记未完成"),
        ("rm", cmd_rm, "删除任务"),
    ]:
        s = sub.add_parser(name, help=helptext, aliases=[
        s.add_argument("id", type=int, help="任务 ID（用 l
```

```
        s = sub.add_parser(name, help=helptext, aliases=[
        s.add_argument("id", type=int, help="任务 ID（用 l
        s.set_defaults(func=func)

    e = sub.add_parser("edit", help="修改内容 / 优先级 / 截
    e.add_argument("id", type=int, help="任务 ID")
    e.add_argument("--text", default=None, help="新的内容"
    e.add_argument("-p", "--priority", choices=["high", "
                   default=argparse.SUPPRESS, help="新的优
    e.add_argument("-d", "--due", type=parse_due,
                   default=argparse.SUPPRESS,
                   help="新的截止日期，传 none 表示清除"
    e.set_defaults(func=cmd_edit)

    c = sub.add_parser("clear", help="清空全部")
    c.set_defaults(func=cmd_clear)

    args = p.parse_args()
    args.func(args)


if __name__ == "__main__":
    main()
```

# 用法示例

bash 复制

```
# 添加：带优先级和截止日期
3    [ ]   中       2026-10-20 週期    預約体位
2    [ ]   低       -                  买牛奶
-------------------------------------------------------
```

```
2     [ ]    低         -                              买牛奶
------------------------------------------------------------
共 3 项，已完成 0 项，未完成 3 项，逾期 1 项
```

bash 复制

```bash
# 修改（可单独改优先级、改截止日期、改内容，或组合）
python todo.py edit 2 -p high
python todo.py edit 2 -d 2026-12-01
python todo.py edit 2 -d none           # 清除截止日期
python todo.py edit 2 --text "买牛奶和鸡蛋"

# 完成 / 撤销 / 删除
python todo.py done 1
python todo.py undone 1
python todo.py rm 2
python todo.py delete 2                 # rm 的别名
python todo.py list -a                  # 含已完成
python todo.py clear                    # 清空
```

# 本次新增点

表格

| 功能 | 实现方式 |
| --- | --- |
| 优先级 | 字段 `priority`，取值 high/medium/low，`list` 中高→低排序，表格显示高/中/低 |

| | |
|---|---|
| | 中/低 |
| **截止日期** | 字段 `due`，`add -d` / `edit -d` 设置，格式校验，传 `none` 清除 |
| **删除** | `rm`（另有 `del` / `delete` 别名），按稳定 ID 删除 |
| **排序与逾期** | 未完成 → 优先级 → 截止日期 → ID；逾期未完成项显示“逾期”并计数 |
| **稳定 ID** | 每个任务带自增 `id`，排序不会打乱 ID，`done/rm/edit` 始终指向同一个任务 |
| **兼容旧数据** | 老 `~/.todo.json` 缺字段会自动补 `priority=medium`、`due=None`、`id` |

需要的话还能再加：按优先级/截止日期**过滤**（`list -p high`）、`--done` 只看已完成、或到期当天提醒。

加一个逾期前三个单位时间主动提醒用户的功能
将每日在细化为每小时

已思考（用时 23 秒） >

理解你的需求：**提醒检查频率从每天一次细化为每小时一次，并且在到期前 3 小时（3 个“小时”单位）主动提醒**。同时保留已有的删除、优先级、截止日期功能。

时保留已有的删除、优先级、截止日期功能。

纯命令行程序要做到“主动提醒”，需要两层机制配合：

1. `remind` **子命令**：一次性检查并输出提醒，挂到 cron / 计划任务上每小时跑一次，实现真正的主动提醒。
2. **惰性检查**：你执行任何其他命令时，如果距上次检查超过 1 小时，顺手检查并提醒，这样即使没配 cron 也不会漏。
3. 截止日期从“只有日期”扩展为**支持精确到小时**（`YYYY-MM-DD HH:MM`），否则没法按小时提醒；纯日期视为当天 23:59 截止。

## 完整代码（替换 `todo.py`）

python 复制

```python
#!/usr/bin/env python3
"""命令行待办清单：优先级 / 截止日期（精确到小时） / 删除 / 每小时

import argparse
import json
import os
import sys
from datetime import datetime, timedelta

DATA_FILE = os.path.join(os.path.expanduser("~"), ".todo.
```

```
DATA_FILE = os.path.join(os.path.expanduser("~"), ".todo.
LAST_CHECK_FILE = os.path.join(os.path.expanduser("~"), "

PRIORITY_ORDER = {"high": 0, "medium": 1, "low": 2}
PRIORITY_LABEL = {"high": "高", "medium": "中", "low"

DEFAULT_LEAD_HOURS = 3       # 到期前 3 小时提醒
REMIND_INTERVAL_HOURS = 1    # 惰性检查最小间隔：每小时一次


# ---------------- 数据层 ----------------

def load_tasks():
    if not os.path.exists(DATA_FILE):
        return []
    try:
        with open(DATA_FILE, "r", encoding="utf-8") as f:
            tasks = json.load(f)
    except (json.JSONDecodeError, OSError):
        return []

    nxt = max((t.get("id") or 0 for t in tasks), defa
    for t in tasks:
        t.setdefault("done", False)
        t.setdefault("priority", "medium")
        t.setdefault("due", None)
        if not t.get("id"):


def save_tasks(tasks):
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(tasks, f, ensure_ascii=False, indent=2)


def find_task(tasks, tid):
```

```
def find_task(tasks, tid):
    for t in tasks:
        if t["id"] == tid:
            return t
    print(f"错误: 没有 ID 为 {tid} 的任务", file=sys.stderr)
    sys.exit(1)


# ---------------- 时间处理 ----------------

def parse_due(s):
    """支持 YYYY-MM-DD HH:MM / YYYY-MM-DDTHH:MM / YYYY-MM
    if s is None:
        return None
    s = s.strip()
    if s.lower() in ("none", "null", "-", ""):
        return None
    for fmt in ("%Y-%m-%d %H:%M", "%Y-%m-%dT%H:%M", "%Y-%
        try:
            return datetime.strptime(s, fmt).strftime(fmt
        except ValueError:
            continue
    raise argparse.ArgumentTypeError(
        f"日期格式应为 YYYY-MM-DD 或 'YYYY-MM-DD HH:MM', 收到
    )


def due_datetime(t):
    """把 due 字段解析为 datetime; 纯日期视为当天 23:59:59"""
    s = t.get("due")
    if not s:
        return None
    for fmt in ("%Y-%m-%d %H:%M", "%Y-%m-%dT%H:%M", "%Y-%
        try:
            dt = datetime.strptime(s, fmt)
            if fmt == "%Y-%m-%d":
```

```
            dt = datetime.strptime(s, fmt)
            if fmt == "%Y-%m-%d":
                dt = dt.replace(hour=23, minute=59, s
            return dt
        except ValueError:
            continue
    return None


def humanize(delta):
    secs = int(abs(delta.total_seconds()))
    h, m = secs // 3600, (secs % 3600) // 60
        elif delta <= timedelta(hours=lead_hours):
            alerts.append((t, "soon", dt))
    alerts.sort(key=lambda x: x[2])
    return alerts


def print_alerts(alerts, lead_hours):
    now = datetime.now()
    print(f"  {'状态':<8} {'任务':<{w}} {'截止':<16} {'还有':>8}")

    alerts = collect_alerts(load_tasks(), DEFAULT_LEA
    update_last_check()
    if alerts:
        print_alerts(alerts, DEFAULT_LEAD_HOURS)
        print()


# ---------------- 命令 ----------------

def cmd_add(args):
    tasks = load_tasks()
    tid = max((t["id"] for t in tasks), default=0) + 1
    tasks.append({
        "id": tid,
        "text": args.text,
```

```
        "text": args.text,
        "done": False,
        "priority": args.priority,
        "due": args.due,
        "created": datetime.now().strftime("%Y-%m-%d %H:%
    })
    save_tasks(tasks)
    due = args.due or "无"
    print(f"已添加 #{tid}: {args.text}（优先级 {PRIORI

def sort_key(t):
    return (
        t["done"],
        due = dt.strftime("%Y-%m-%d %H:%M") + "  逾期
    else:
        due = dt.strftime("%Y-%m-%d %H:%M")
    print(f"{t['id']:<5}{mark:<6}{pr:<8}{due:<18}{t['
print("-" * 60)

def cmd_remind(args):
    alerts = collect_alerts(load_tasks(), args.lead)
    update_last_check()
    if not alerts:
        if not args.quiet:
            print(f"(无) 未来 {args.lead} 小时内没有即将到期

def cmd_undone(args):
    tasks = load_tasks()
    t = find_task(tasks, args.id)
    t["done"] = False
    save_tasks(tasks)
    print(f"已重置 #{t['id']} {t['text']}")

a.set_defaults(func=cmd_add)
```

```
    a.set_defaults(func=cmd_add)

    l = sub.add_parser("list", help="列出待办（按优先级+截止
    l.add_argument("-a", "--all", action="store_true", he
    l.set_defaults(func=cmd_list)

    r = sub.add_parser("remind", help="立即检查并输出提
    r.add_argument("--lead", type=int, default=DEFAU
                   help=f"提前多少小时提醒，默认 {DEFAULT_LE
```

```
        s.set_defaults(func=func)

    e = sub.add_parser("edit", help="修改内容 / 优先级 / 截
    e.add_argument("id", type=int, help="任务 ID")
    e.add_argument("--text", default=None, help="新的内容"
    e.add_argument("-p", "--priority", choices=["high", "
                   default=argparse.SUPPRESS, help="新的优
    e.add_argument("-d", "--due", type=parse_due, default
                   help="新的截止时间，传 none 清除")
    e.set_defaults(func=cmd_edit)

    c = sub.add_parser("clear", help="清空全部")
    c.set_defaults(func=cmd_clear)

    args = p.parse_args()
    maybe_lazy_remind(args.cmd)    # 每小时最多一次的惰性提醒
    args.func(args)


if __name__ == "__main__":
    main()
```

# 用法示例

bash 复制

```
# 截止时间可精确到小时
python todo.py add "交季度报告" -p high -d "2026-11-01
python todo.py add "预约体检"    -p medium -d "2026-10-2
python todo.py add "买牛奶"      -p low
```